

CREATE NEW API EXITPOLL

Contents

1) Create Entity to ExitPoll.Domain/Entities	4
• Description.....	4
• Code Example	4
2) Add Entity to DbContext (ExitPoll.Infrastructure/Data/ApplicationDbContext.cs).4	
• Description.....	4
• Code Example	5
3) Create Migration & Update Database	5
• Description.....	5
• Commands.....	5
4) Create Repository Interface (ExitPoll.Domain/Interfaces)	5
• Description	5
• Code Example	6
4) Create DTO (ExitPoll.Application/DTOs).....	6
• Description	6
• Code Example	6
6) Create Service Interface (ExitPoll.Application/Interfaces)	6
• Description.....	6
• Code Example	7
7) Create Service Implementation (ExitPoll.Application/Services)	7
• Description.....	7
• Code Example	7
8) Create Repository Implementation (ExitPoll.Infrastructure/Repositories)	9
• Description.....	9
• Code Example	9
9) Register Dependencies (ExitPoll.API/Program.cs).....	10
• Description	10
• Code Example	10
10) Create Controller (ExitPoll.API/Controllers).....	10

• Description.....	10
• Code Example	11
Full Request Flow.....	13

1) Create Entity to ExitPoll.Domain/Entities

- **Description**

What is this?

This is the **core database model**. It represents a real table in the database and defines the structure of your data.

Why do we need it?

It is the **source of truth** for your data structure and is used across repository and service layers.

Who uses it?

- Infrastructure (EF Core DbContext)
- Repository layer
- Service layer (indirectly via repository)

- **Code Example**

```
using System.ComponentModel.DataAnnotations.Schema;

namespace ExitPoll.Domain.Entities
{
    public class EntityName
    {
        public int Id { get; set; }
        public string Name { get; set; } //Attributes
    }
}
```

File name must match entity: EntityName.cs

2) Add Entity to DbContext

(ExitPoll.Infrastructure/Data/ApplicationDbContext.cs)

- **Description**

What is this?

- This registers the entity with **Entity Framework Core**, so it becomes a database table.

Why do we need it?

- Without this, EF Core cannot track or create the table in the database.

Who uses it?

- EF Core (runtime)
- Migrations system
- **Code Example**

```
public DbSet<EntityName> EntityName { get; set; }
```

Note:

 - EntityName = C# class
 - EntityNames = database table name (convention or pluralized form)

3) Create Migration & Update Database

- **Description**

What is this?

- This is how we physically apply schema changes to the database.

Why do we need it?

- It keeps database structure synchronized with your code.

Who uses it?

- EF Core CLI / Package Manager Console
- Database engine

- **Commands**

1. add-migration name of migration *//should be unique*
2. update-database

4) Create Repository Interface (ExitPoll.Domain/Interfaces)

- **Description**

What is this?

- This defines **data access contracts** (CRUD operations).

Why do we need it?

- To separate business logic from database logic (clean architecture rule).

Who uses it?

- Application services (business logic layer)
- Infrastructure implementation (EF Core)

- **Code Example**

```
using ExitPoll.Domain.Entities;

namespace ExitPoll.Domain.Interfaces
{
    public interface IEntityNameRepository
    {
        Task<IEnumerable<EntityName>> GetAllAsync();
        Task<EntityName?> GetByIdAsync(int id);
        Task AddAsync(EntityName EntityName);
        Task UpdateAsync(EntityName EntityName);
        Task DeleteAsync(int id);
    }
}
```

4) Create DTO (ExitPoll.Application/DTOs)

- **Description**

What is this?

- A DTO (Data Transfer Object) is a **safe data model used between API and frontend**.

Why do we need it?

- We never expose database entities directly to API clients.

Who uses it?

- API controllers
- Application services

- **Code Example**

```
namespace ExitPoll.Application.DTOs
{
    public class EntityNameDto
    {
        public int Id { get; set; }
        public string Name { get; set; }
    }
}
```

6) Create Service Interface (ExitPoll.Application/Interfaces)

- **Description**

What is this?

- Defines **business logic contracts** (what the system can do).

Why do we need it?

- Controllers depend on this, not repositories directly → keeps logic clean and testable.

Who uses it?

- API Controllers
- Service implementation

- **Code Example**

```
using ExitPoll.Application.DTOS;

namespace ExitPoll.Application.Interfaces
{
    public interface IEntityNameService
    {
        Task<IEnumerable<EntityNameDto>> GetAllAsync();
        Task<EntityNameDto?> GetByIdAsync(int id);
        Task<int> CreateAsync(EntityNameDto dto);
        Task UpdateAsync(EntityNameDto dto);
        Task DeleteAsync(int id);
    }
}
```

7) Create Service Implementation (ExitPoll.Application/Services)

- **Description**

What is this?

- This is where **business logic lives** and mapping between Entity ↔ DTO happens.

Why do we need it?

- Keeps controllers thin and separates business logic from HTTP layer.

Who uses it?

- API Controllers

Depends on:

- Repository Interface (Domain layer)

- **Code Example**

```
using ExitPoll.Application.DTOS;
```

```

using ExitPoll.Domain.Interfaces;
using ExitPoll.Domain.Entities;
using ExitPoll.Application.Interfaces;

namespace ExitPoll.Application.Services
{
    public class EntityNameService : IEntityNameService
    {
        private readonly IEntityNameRepository _repository;

        public EntityNameService(IEntityNameRepository repository)
        {
            _repository = repository;
        }

        public async Task<IEnumerable<EntityNameDto>> GetAllAsync()
        {
            var entities = await _repository.GetAllAsync();
            return entities.Select(e => new EntityNameDto
            {
                Id = e.Id,
                Name = e.Name
            });
        }

        public async Task<EntityNameDto?> GetByIdAsync(int id)
        {
            var entity = await _repository.GetByIdAsync(id);
            if (entity == null) return null;

            return new EntityNameDto
            {
                Id = entity.Id,
                Name = entity.Name
            };
        }

        public async Task<int> CreateAsync(EntityNameDto dto)
        {
            var entity = new EntityName
            {
                Name = dto.Name
            };

            await _repository.AddAsync(entity);
            return entity.Id;
        }

        public async Task UpdateAsync(EntityNameDto dto)
        {
            var entity = new EntityName
            {
                Id = dto.Id,
                Name = dto.Name
            };

            await _repository.UpdateAsync(entity);
        }

        public async Task DeleteAsync(int id)
        {

```

```

        }
        await _repository.DeleteAsync(id);
    }
}

```

8) Create Repository Implementation (ExitPoll.Infrastructure/Repositories)

- **Description**

What is this?

- This is the **database implementation using EF Core**.

Why do we need it?

- It connects your application to the database.

Who uses it?

- Application Service layer

Depends on:

- DbContext

- **Code Example**

```

using ExitPoll.Domain.Entities;
using ExitPoll.Domain.Interfaces;
using ExitPoll.Infrastructure.Data;
using Microsoft.EntityFrameworkCore;

namespace ExitPoll.Infrastructure.Repositories
{
    public class EntityNameRepository : IEntityNameRepository
    {
        private readonly ApplicationDbContext _db;

        public EntityNameRepository(ApplicationDbContext db)
        {
            _db = db;
        }

        public async Task<IEnumerable<EntityName>> GetAllAsync()
            => await _db.EntityNames.ToListAsync();

        public async Task<EntityName?> GetByIdAsync(int id)
            => await _db.EntityNames.FindAsync(id);

        public async Task AddAsync(EntityName entityName)
        {
            _db.EntityNames.Add(entityName);
            await _db.SaveChangesAsync();
        }

        public async Task UpdateAsync(EntityName entityName)

```

```

        {
            _db.EntityNames.Update(entityName);
            await _db.SaveChangesAsync();
        }

        public async Task DeleteAsync(int id)
        {
            var entity = await _db.EntityNames.FindAsync(id);
            if (entity == null) return;
            _db.EntityNames.Remove(entity);
            await _db.SaveChangesAsync();
        }
    }
}

```

9) Register Dependencies (ExitPoll.API/Program.cs)

- **Description**

What is this?

- Dependency Injection (DI) configuration.

Why do we need it?

- It tells ASP.NET Core how to resolve interfaces at runtime.

Who uses it?

- Runtime container (ASP.NET Core DI system)

- **Code Example**

- ```

// Repositories
builder.Services.AddScoped<IEntityNameRepository, EntityNameRepository>();

Services
builder.Services.AddScoped<IEntityNameService, EntityNameService>();

```

## 10) Create Controller (ExitPoll.API/Controllers)

- **Description**

### What is this?

- This is the **HTTP entry point** of your API.

## Why do we need it?

- It exposes endpoints for frontend or external systems.

## Who uses it?

- Frontend (React/Angular/mobile apps)
- External clients

## Flow

Controller → Service → Repository → Database

## Key Points:

- Controllers do NOT contain business logic
- Only call services
- Return HTTP responses

## • Code Example

```
using ExitPoll.Application.DTOS;
using ExitPoll.Application.Interfaces;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

// For more information on enabling Web API for empty projects, visit
https://go.microsoft.com/fwlink/?LinkID=397860

namespace ExitPoll.API.Controllers
{
 [Route("api/[controller]")]
 [ApiController]
 public class EntityNamesController : ControllerBase
 {
 private readonly IEntityNameService _EntityNameService;

 public EntityNamesController(IEntityNameService EntityNameService)
 {
 _EntityNameService = EntityNameService;
 }

 // GET: api/<EntityNamesController>
 [HttpGet]
 [Authorize(Roles = "Admin,Agent")]
 public async Task<IActionResult> Get()
 {
 var EntityNames = await _EntityNameService.GetAllAsync();
 return Ok(EntityNames);
 }

 // GET api/<EntityNamesController>/5
 [Authorize(Roles = "Admin,Agent")]
 [HttpGet("{id}")]
 }
}
```

```

public async Task<IActionResult> Get(int id)
{
 var EntityName = await _EntityNameService.GetByIdAsync(id);
 return Ok(EntityName);
}

[HttpPost]
[Authorize(Roles = "Admin")]
public async Task<IActionResult> Post([FromBody] EntityNameDto dto)
{
 if (!ModelState.IsValid) return BadRequest(ModelState);

 var id = await _EntityNameService.CreateAsync(dto);

 return CreatedAtAction(nameof(Get), new { id }, id);
}

// PUT api/<EntityNamesController>/5
[HttpPut("{id}")]
[Authorize(Roles = "Admin")]
dto) public async Task<IActionResult> Put(int id, [FromBody] EntityNameDto
{
 if (id != dto.Id) return BadRequest();

 await _EntityNameService.UpdateAsync(dto);
 return NoContent();
}

// DELETE api/<EntityNamesController>/5
[HttpDelete("{id}")]
[Authorize(Roles = "Admin")]
public async Task<IActionResult> Delete(int id)
{
 await _EntityNameService.DeleteAsync(id);
 return NoContent();
}
}
}

```

## Full Request Flow

**Frontend / Client**

↓

**Controllers (ExitPoll.API) -> EntityNamesController.cs**

↓

**Service Interface (ExitPoll.Application) -> IEntityNameService.cs**

↓

**Service Implementation (ExitPoll.Application) -> EntityNameService.cs**

↓

**Repository Interface (ExitPoll.Domain) -> IEntityNameRepository.cs**

↓

**Repository Implementation (ExitPoll.Infrastructure) -> EntityNameRepository.cs**

↓

**DbContext (ExitPoll.Infrastructure) -> ApplicationDbContext.cs**

↓

**Database (SQL Server)**

### 1. API Layer (Entry Point)

ExitPoll.API/Controllers/EntityNamesController.cs

- Receives HTTP requests
- Calls service layer
- Returns HTTP response

### 2. Application Service Contract

ExitPoll.Application/Interfaces/IEntityNameService.cs

- Defines business operations (CRUD rules)

- Used by controller

### 3. Application Service Logic

ExitPoll.Application/Services/EntityNameService.cs

- Contains business logic
- Maps DTO ↔ Entity
- Calls repository

### 4. Domain Repository Contract

ExitPoll.Domain/Interfaces/IEntityNameRepository.cs

- Defines data access operations
- No EF Core, no database logic here

### 5. Infrastructure Repository Implementation

ExitPoll.Infrastructure/Repositories/EntityNameRepository.cs

- Implements repository using EF Core
- Talks directly to DbContext

### 6. Database Context

ExitPoll.Infrastructure/Data/ApplicationDbContext.cs

- EF Core mapping layer
- Contains:

```
public DbSet<EntityName> EntityNames { get; set; }
```

### 7. Database

- SQL Server / PostgreSQL / etc.
- Stores actual tables:
  - EntityNames

## **Clean Mental Model (Very Important)**

Think of it like this:

- **Controller = Door**
- **Service = Brain**
- **Repository = Hands**
- **DbContext = Translator to database**
- **Database = Storage**